App Mapping and Navigation

App Mapping and Navigation

1. Course Content
2. Preparation
4. App Mapping Case
5. App Navigation Case
6. Source Code Analysis

1. Course Content

Control the car's mapping and navigation through the mobile app 【ROS Robot】.

2. Preparation

1. Your phone needs to first download the `【ROS Robot】` app. Android/iOS phone users, please scan the QR code to download the remote control software. iOS users can also search for and download the mapping and navigation app `【ROSRobot】` in the App Store.



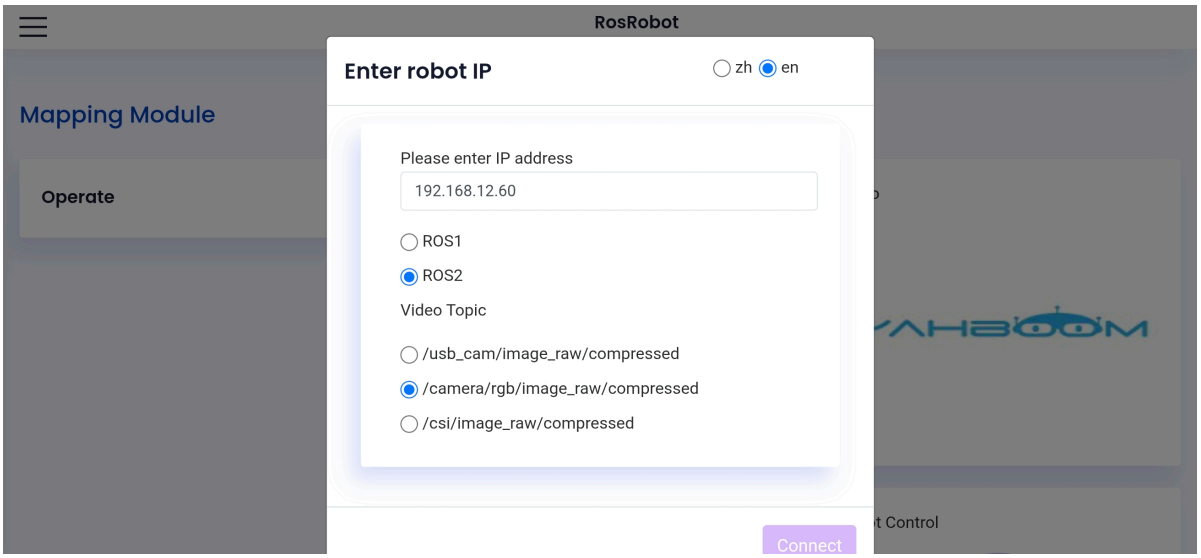
2. Then the car and the phone must be on the same local area network; this can be achieved by connecting to the same Wi-Fi.

4. App Mapping Case

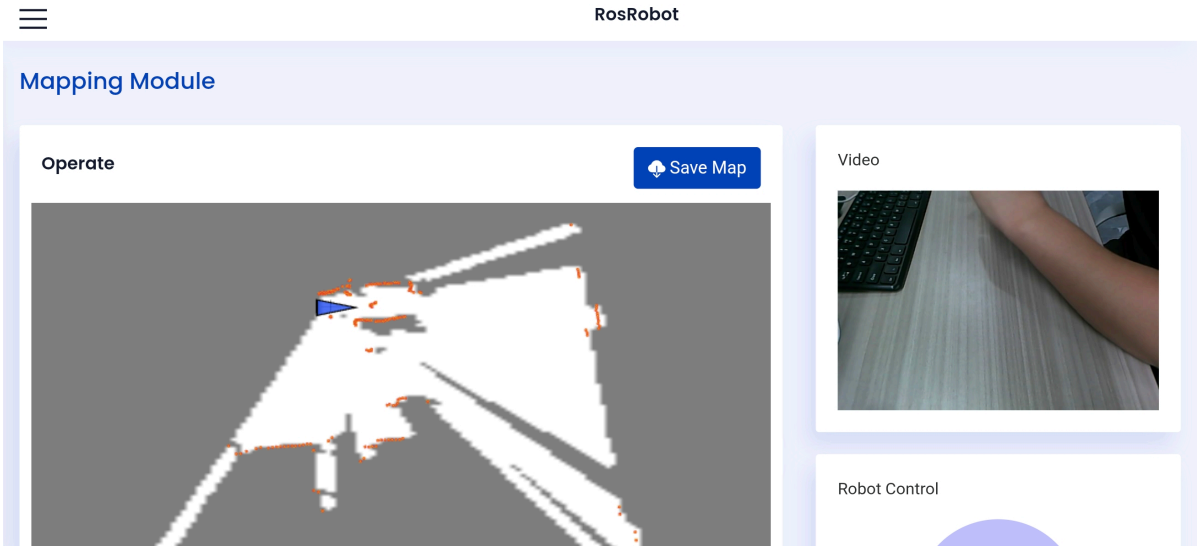
- Enter the following command in the car's terminal to start the car's app mapping service node

```
ros2 launch yahboomcar_bringup bringup_app.launch.py
```

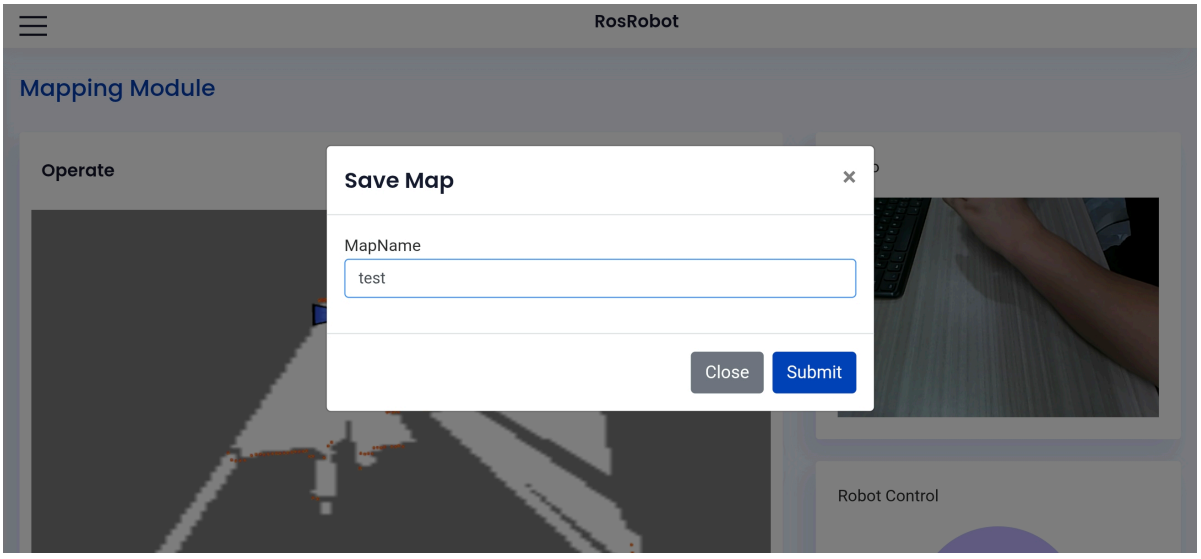
- Open `【ROS Robot】`, which displays as shown below
- Enter the car's IP address ——> `ROS2` ——> `VideoTopic/camera/rgb/image_raw/compressed` ——> 【Connect】



- After successfully connecting, it displays as follows



- Use the sliding wheel to control the car to slowly move through the area to be mapped, then click Save Map, enter a map name and click Submit to save the map



- The map will be automatically saved in the system's `~/map` directory, and the terminal will indicate that the map was saved successfully

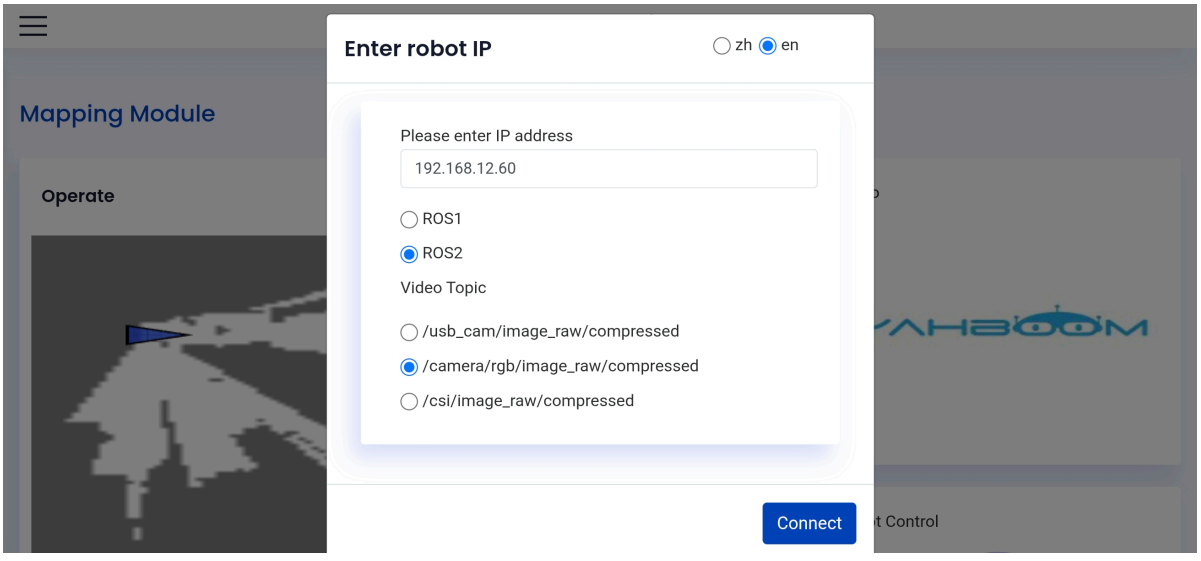
```
[server-6] [INFO] [1765334967.910089971] [map_saver]: Creating
[server-6] [INFO] [1765334967.912193848] [map_saver]: Configuring
[server-6] [INFO] [1765334967.922756528] [map_saver]: Saving map from 'map' topic to '/home/jetson/map
/4444' file
[server-6] [WARN] [1765334967.922858419] [map_saver]: Free threshold unspecified. Setting it to defaul
t value: 0.250000
[server-6] [WARN] [1765334967.922887156] [map_saver]: Occupied threshold unspecified. Setting it to de
fault value: 0.650000
[server-6] [WARN] [1765334967.951984200] [map_io]: Image format unspecified. Setting it to: pgm
[server-6] [INFO] [1765334967.954177999] [map_io]: Received a 449 X 224 map @ 0.05 m/pix
[server-6] [INFO] [1765334968.016817320] [map_io]: Writing map occupancy data to /home/jetson/map/4444
.pgm
[server-6] [INFO] [1765334968.019211798] [map_io]: Writing map metadata to /home/jetson/map/4444.yaml
[server-6] [INFO] [1765334968.019438653] [map_io]: Map saved
[server-6] [INFO] [1765334968.019460606] [map_saver]: Map saved successfully
[server-6] [INFO] [1765334968.023690856] [map_saver]: Destroying
```

5. App Navigation Case

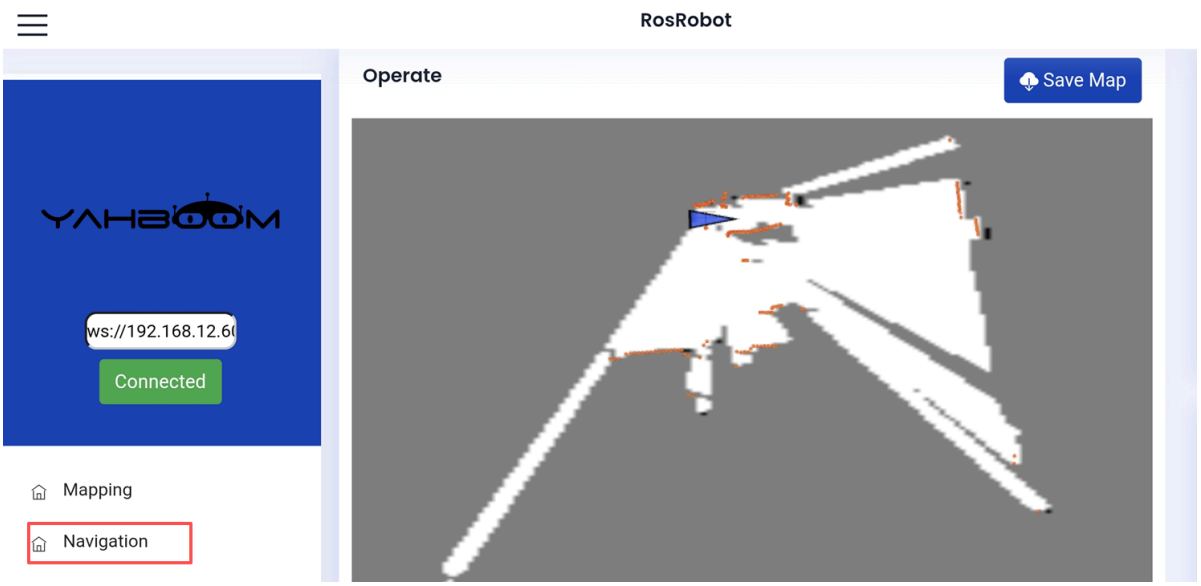
```
ros2 launch yahboomcar_bringup bringup_app.launch.py navigation_mode:=True
```

> **Note**
>
> **Optional parameter description**
>
> - map_file
>   - Map file path; here the map yaml name is required; the default value is `map.yaml`

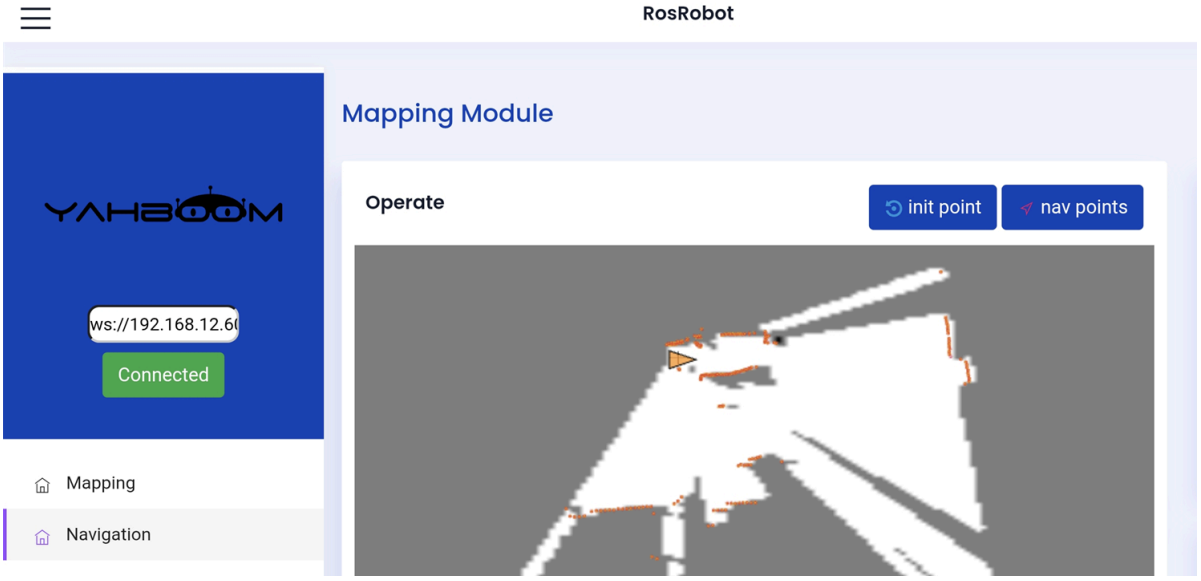
- Open `【ROS Robot】`, which displays as shown below
- Enter the car's IP address ——> `ROS2` ——> `VideoTopic/camera/rgb/image_raw/compressed` ——> 【Connect】



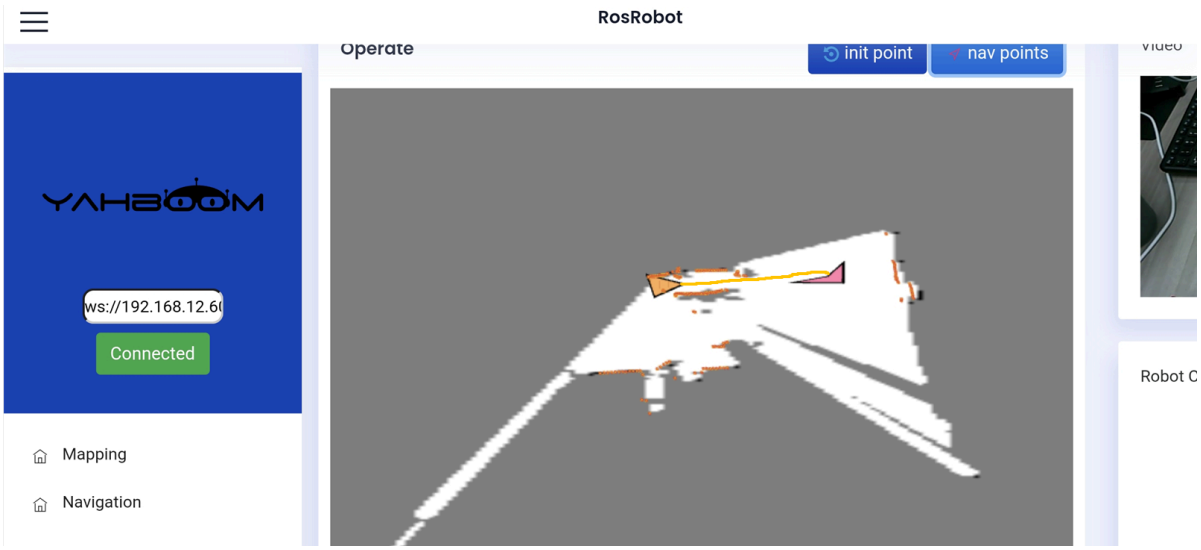
- As shown below, select the navigation interface:



- Then, based on the car's actual pose, click 【Set Initial Pose】 to give the car an initial pose. If the area scanned by the LiDAR roughly overlaps with the actual obstacle, it means the pose is accurate. As shown below,



- Click 【Set Navigation Point】 to give the car a destination; the car will plan a path and move to the destination along the path. As shown below:



6. Source Code Analysis

Launch and backend service source code paths:

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/yahboomcar_bringup/launch/bringup_app.launch.py
```

```
~/yahboomcar_ros2_ws/yahboomcar_ws/src/ros_robot_app/ros_robot_app/
```

- The key startup program is as follows:
- The program uses the two launch parameters slam_mode and navigation_mode to decide between the two startup modes of mapping or navigation, and starts the rosbridge WebSocket server to transmit relevant data through the WebSocket protocol.

```
    slam_mode=LaunchConfiguration('slam_mode', default="slam_toolbox")
    navigation_mode=LaunchConfiguration('navigation_mode', default="False")


    #gmapping
    gmapping_launch=IncludeLaunchDescription(
        PythonLaunchDescriptionSource(os.path.join(yahboomcar_bringup_dir,
                        'launch',
                        'gmapping.launch.py')),
        launch_arguments={
                        "rviz": "False",
                         }.items(),
        condition=IfCondition(PythonExpression(["'", slam_mode, "' == 'gmapping'
and '", navigation_mode, "' == 'False' "]))
    )
    #slam_toolbox
    slam_toolbox_launch=IncludeLaunchDescription(
        PythonLaunchDescriptionSource(os.path.join(yahboomcar_bringup_dir,
                        'launch',
```

```
                        'slam_toolbox.launch.py')),
        launch_arguments={
                        "rviz": "False",
                         }.items(),
        condition=IfCondition(PythonExpression(["'", slam_mode, "' ==
'slam_toolbox' and '", navigation_mode, "' == 'False' "]))
    )

    #Start Cartographer mapping mode.
    cartographer_launch=IncludeLaunchDescription(
        PythonLaunchDescriptionSource(os.path.join(yahboomcar_bringup_dir,
                        'launch',
                        'cartographer.launch.py')),
        launch_arguments={
                        "rviz": "False",
        }.items(),
        condition=IfCondition(PythonExpression(["'", slam_mode, "' ==
'cartographer' and '", navigation_mode, "' == 'False' "]))

        )

    #Start the rosbridge node.
    bringup_rosbridge_server = ExecuteProcess(
        cmd=['ros2', 'launch', 'rosbridge_server',
'rosbridge_websocket_launch.xml'],
        output='screen'
    )

    transmission_node = Node(
            package='ros_robot_app',
            executable='transmission',
            name='transmission',
            parameters=[
                {'rgb_topic': GetCameraRgbTopic()},
                {'flip_laser_points': flip_laser_points}
            ],
            output='screen',
    )


    #Robot pose publishing node
    robot_pose_publisher=Node(
        package="ros_robot_app",
        executable="robot_pose_publisher",
        name="robot_pose_publisher",
        output="screen",
        parameters=[
                {"use_sim_time": False},
                {"is_stamped": True},
                {"map_frame": "map"},
                {"base_frame": "base_link"}
        ]
    )

    #App map saving service node
    app_save_map_node = Node(
```

```
        package='ros_robot_app',
        executable="app_save_map",
        name="app_save_map",
        output="screen",
    )

    app_nav= IncludeLaunchDescription(

 PythonLaunchDescriptionSource(os.path.join(yahboomcar_bringup_dir,
                                    "launch",
                                    'nav2.launch.py')),
                launch_arguments={
                    "map": LaunchConfiguration("map_file"),
                    "rviz": "False",
                    "localization_mode": "amcl",
                    "slam": "False",
                }.items(),
                condition=IfCondition(PythonExpression(["'", navigation_mode, "'
== 'True'" ]))
             )

    return LaunchDescription([
        *declared_arguments,
        bringup_camera,
        app_nav,                                #app navigation node
        bringup_rosbridge_server,               #rosbridge
        transmission_node,                      #publish rgb image and laser scan
point
        robot_pose_publisher,                   #robot pose publishing node
        gmapping_launch,                        #gmapping slam
        slam_toolbox_launch,                    #slam_toolbox slam
        cartographer_launch,                    #cartographer slam
        app_save_map_node
    ])
```